EXPRESIONES REGULARES EN PHP Y LARAVEL

Índice

Contenido

1 PHP 2

1.1 Patrón 2

1.2 Caracteres especiales 2

1.3 Cuantificadores de repetición 3

1.4 Agrupación Corchetes [] 3

1.5 Agrupación con Paréntesis () 5

1.6 Alternancia | 5

1.7 Modificadores 5

2 Laravel 5

1 PHP

Todas las expresiones regulares de PHP deben estar delimitadas. El delimitador más utilizado es / aunque también puedes usar #, ~, %, etc.

preg_match() → Busca un patrón en una cadena

Devuelve 1 si hay coincidencia, 0 si no hay, y false si hay un error.

```php
$cadena = "Hola, mi correo es usuario@example.com";
$patron = "/[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}/i";

if (preg_match($patron, $cadena, $coincidencias)) {
    echo "Correo encontrado: " . $coincidencias[0];
} else {
    echo "No se encontró un correo";
}
```

1.1 Patrón

```php
$patron = "/expresion_regular/modificadores";
```

```php
$patron = "/hola/";  // Coincide con la palabra "hola"
```

1.2 Caracteres especiales

Metacarácter	Significado	Ejemplo
`.`	Cualquier carácter	`/h.t/` → Coincide con "hat", "hit", "hot"
`\d`	Dígito (`0-9`)	`/\d+/` → Encuentra "123" en "abc 123"
`\w`	Letra, número o `_`	`/\w+/` → Encuentra "Hola" en "Hola mundo"
`\s`	Espacio en blanco	`/\s+/` → Encuentra espacios, tabulaciones o saltos de línea
`^`	Inicio de cadena	`/^Hola/` → Coincide solo si la cadena empieza con "Hola"
`$`	Fin de cadena	`/mundo$/` → Coincide si la cadena termina en "mundo"
`\b`	Límite de palabra	`/\bPHP\b/` → Coincide solo con la palabra "PHP" completa

```php
$patron = "/\d+/"; // Encuentra números enteros
$texto = "Tengo 25 años";
preg_match($patron, $texto, $coincidencias);
print_r($coincidencias); // ["25"]
```

1.3 Cuantificadores de repetición

Cuantificador	Significado	Ejemplo
`*`	0 o más veces	`/a*/` → "aaa", "a", ""
`+`	1 o más veces	`/a+/` → "aaa", "a", pero no ""
`?`	0 o 1 vez (opcional)	`/a?/` → "a" o ""
`{n}`	Exactamente `n` veces	`/\d{3}/` → "123" en "abc 12345"
`{n,}`	`n` o más veces	`/\d{2,}/` → "123" en "abc 123"
`{n,m}`	Entre `n` y `m` veces	`/\d{2,4}/` → "123" en "abc 12345"

```php
$patron = "/^\d{5}$/";
$codigo = "28080";
if (preg_match($patron, $codigo)) {
    echo "Código válido";
} else {
    echo "Código inválido";
}
```

1.4 Agrupación Corchetes []

Permiten definir un grupo de caracteres entre los que **puede coincidir uno solo**.

```php
php

$patron = "/[aeiou]/"; // Coincide con cualquier vocal
$cadena = "Hola";
if (preg_match($patron, $cadena)) {
    echo "Contiene una vocal"; // ✅ Coincide
}
```

Se pueden usar rangos

Patrón	Descripción
`[a-z]`	Cualquier letra minúscula
`[A-Z]`	Cualquier letra mayúscula
`[0-9]`	Cualquier número del 0 al 9
`[a-zA-Z]`	Cualquier letra mayúscula o minúscula
`[a-zA-Z0-9]`	Cualquier letra o número

🔹 Ejemplo: Validar que una cadena solo tenga letras y números

```php
$patron = "/^[a-zA-Z0-9]+$/";
$cadena = "Usuario123";
if (preg_match($patron, $cadena)) {
    echo "Cadena válida"; // ✅ Coincide
} else {
    echo "Cadena inválida";
}
```

Si colocamos `^` al inicio dentro de `[]`, significa "**ninguno de estos caracteres**".

Patrón	Descripción
`[^aeiou]`	Cualquier carácter que **NO** sea una vocal
`[^0-9]`	Cualquier carácter que **NO** sea un número

🔹 Ejemplo: Buscar un carácter que no sea número

```php
$patron = "/[^0-9]/";
$cadena = "123A";
if (preg_match($patron, $cadena)) {
    echo "Contiene un carácter que no es número"; // ✅ Coincide por la "A"
}
```

Puedes combinar `[]` con repetición:

Patrón	Significado
`[0-9]+`	Uno o más números
`[A-Z]{3}`	Exactamente 3 letras mayúsculas
`[a-zA-Z]*`	Cero o más letras

🔹 Ejemplo: Validar un código de producto con 3 letras y 4 números (ej. ABC1234)

```php
$patron = "/^[A-Z]{3}[0-9]{4}$/";
$codigo = "XYZ2024";
if (preg_match($patron, $codigo)) {
    echo "Código válido"; // ✅ Coincide
} else {
    echo "Código inválido";
}
```

1.5 Agrupación con Paréntesis ()

Elemento	Significado	Ejemplo
`( )`	Agrupa expresiones	`/(abc)+/` → "abcabc"

1.6 Alternancia |

El patrón `rojo|azul|verde` buscará **cualquiera de esas palabras** en la cadena.

php Copiar Editar

```php
$patron = "/rojo|azul|verde/";
$cadena = "Tengo un coche azul.";
if (preg_match($patron, $cadena)) {
    echo "Se encontró un color"; // ✅ Coincide con "azul"
}
```

1.7 Modificadores

Los modificadores se escriben después del delimitador de cierre (/patrón/i).

Modificador	Significado
`i`	No distingue mayúsculas de minúsculas
`m`	Trata los saltos de línea como inicio y fin de línea
`s`	Hace que `.` también coincida con saltos de línea

```php
$patron = "/php/i";
$cadena = "Me gusta Php";
if (preg_match($patron, $cadena)) {
    echo "Coincidencia encontrada"; // ✅ Coincide aunque "Php" tenga mayúscula
}
```

2 Laravel

El campo bajo validación debe coincidir con la expresión regular dada. Internamente, esta regla utiliza la función **preg_match** de PHP. El patrón especificado debe obedecer el mismo formato requerido por **preg_match** y, por lo tanto, también incluir delimitadores válidos. Por ejemplo: ***'email' => 'regex:/^.+@.+$/i'***.

Al usar los patrones regex / not_regex, puede ser necesario especificar reglas en un array en lugar de usar delimitadores |, especialmente si la expresión regular contiene un carácter |.